Отчёт по лабораторной работе №21

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы **М8О-114БВ-25 Миронова Татьяна Михайловна** № по списку 12.

Контакты: tatirona@gmail.com

Работа выполнена: «16» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «17» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Программирование на интерпретируемых командных языках ОС UNIX.

2. **Цель работы:** Разработать скрипт для командного интерпретатора Bash и программу на языке Python для генерации заданного числа копий указанного файла с заданным правилом для генерации имён копий.

3. **Задание:** 12. Генерация заданного числа копий указанного файла. Имена копий генерировать добавлением к имени исходного файла следующей по порядку буквы или цифры, начиная с заданной буквы или цифры.

4. **Оборудование**: *Оборудование ПЭВМ студента, если использовалось:*
Процессор**:** Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ

5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось**:***
Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.
Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1.
Язык программирования: Python, Bash

6**. Идея, метод, алгоритм** *решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:*
Идея:
Программа создаёт заданное количество копий указанного файла, автоматически генерируя имена путём добавления к исходному имени последовательных букв или цифр.
Метод:
В зависимости от типа начального символа (цифра или буква) выбирается соответствующий режим генерации имён. В буквенном режиме при достижении границы алфавита ('z' или 'Z') происходит перенос с добавлением дополнительной буквы (аналогично системам счисления).
Алгоритм:

1. Проверка аргументов:
   1.1. Если передан ключ -h или --help → вывести справку и завершить работу.
   1.2. Если количество аргументов не равно 3 → вывести справку и завершить с ошибкой.
2. Проверка входных данных:
   2.1. Присвоить переменным: source_filename = первый аргумент, count = второй аргумент, start_char = третий аргумент.
   2.2. Если start_char не является одним символом и не является числом → вывести ошибку и завершить с кодом 2.
   2.3. Если файл source_filename не существует → вывести ошибку и завершить с кодом 3.
3. Чтение исходного файла:
   3.1. Открыть source_filename на чтение.
   3.2. Считать содержимое в переменную file_text.

4. Определение базового имени и расширения:
   4.1. Найти позицию последней точки в source_filename.
   4.2. base_name = часть до последней точки, extension = часть с последней точкой (включая её).
5. Генерация копий:

Если start_char является числом (цифровой режим):
Для i от 0 до count-1:
- new_name = base_name + (start_char + i) + extension
- Создать файл с именем new_name и записать в него file_text

Иначе (буквенный режим):
carry = "" (счётчик переносов)
offset = 0 (смещение от начального символа)
Для i от 0 до count-1:
- current_ascii = код ASCII(start_char) + offset
- Если достигнут код 'z'+1 или 'Z'+1:
  - Если достигнут 'z'+1 → carry += 'z'
  - Иначе → carry += 'Z'
  - offset = offset - 26
  - current_ascii = current_ascii - 26
- current_char = символ с кодом current_ascii
- new_name = base_name + carry + current_char + extension
- Создать файл с именем new_name и записать в него file_text
- offset = offset + 1

6. Завершение работы.

7. **Сценарий выполнения работы** *(план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию)*.

План работы:
1. Анализ задания и определение требований к программе
2. Проектирование алгоритма работы
3. Написание чернового варианта программы на Python
4. Отладка и тестирование программы
5. Написание альтернативной версии на Bash для сравнения
6. Проведение тестирования с различными входными данными
7. Оформление отчета

Первоначальный текст программы (черновик):

```
import sys

# Простая программа без сложных импортов
# Всё вводится с клавиатуры

print("Программа создания копий файла")
source = input("Введите имя исходного файла: ")
count = int(input("Введите количество копий: "))
start = input("Введите начальный символ (цифру или букву): ")

# Проверка существования файла
try:
    with open(source, 'r') as f:
        content = f.read()
except FileNotFoundError:
    print(f"Ошибка: файл {source} не найден")
    sys.exit(1)

# Разделение имени и расширения
if '.' in source:
    dot_pos = source.rfind('.')
    name = source[:dot_pos]
    ext = source[dot_pos:]
```

```
else:
    name = source
    ext = ''

# Создание копий
if start.isdigit():
    # Цифровой режим
    num = int(start)
    for i in range(count):
        new_name = name + str(num + i) + ext
        with open(new_name, 'w') as f:
            f.write(content)
        print(f"Создан файл: {new_name}")
else:
    # Буквенный режим (упрощённый, без переносов)
    for i in range(count):
        new_char = chr(ord(start) + i)
        new_name = name + new_char + ext
        with open(new_name, 'w') as f:
            f.write(content)
        print(f"Создан файл: {new_name}")
```

Тесты и соображения по тестированию:

Тест	Входные данные	Ожидаемый результат	Примечание
1	test1.txt, 3, a	test1a.txt, test1b.txt, test1c.txt	Проверка буквенного режима
2	test1.txt, 3, 1	test11.txt, test12.txt, test13.txt	Проверка цифрового режима
3	test1.txt, 30, a	test1a.txt ... test1z.txt, test1za.txt ...	Проверка переноса (нужно доработать)
4	test1.txt, 3, ab	Сообщение об ошибке	Проверка валидации
5	несущ.txt, 3, a	Сообщение об ошибке	Проверка существования файла

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. **Подпись преподавателя _____**

8. **Распечатка протокола** *(подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).*

Листинг окончательного варианта программы (Bash):

```
#!/bin/bash

help() {
echo "lab21 [OPTIONS] source_filename count start_char"
echo "Generation of a set number of copies of the specified file."
echo "Copy names are generated by adding the next letter or number to the name of"
echo "the source file, starting with the specified letter or number."
echo "Options:"
echo "  -h, --help  show this help message and exit"
}

if [[ "$1" == "-h" || "$1" == "--help" ]]; then
    help
    exit 0
fi
if [ "$#" -ne 3 ]; then
    help
    exit 1
fi

source_filename="$1"
count="$2"
start_char="$3"

if [[ ${#start_char} -ne 1 ]] && ! [[ $start_char =~ ^[0-9]+$ ]]; then
    echo "Error: Starting letter or digit for file names must be a single character or a digit."
    exit 2
fi
```

```
if [ ! -f "$source_filename" ]; then
    echo "Error: file '$source_filename' does not exist."
    exit 3
fi
if [[ $start_char =~ ^[0-9]+$ ]]; then
    while [ $count -gt 0 ]; do
        count=$(($count - 1))
        new_filename="${source_filename%.*}${start_char}${source_filename##*.}"
        cp "$source_filename" "$new_filename"
        start_char=$((start_char + 1))
    done
else
    plus=""
    ii=0
    for ((i=0; i<count; i++)); do
        current_char=$(echo -n "$start_char" | od -An -tuC)
        current_ascii=$((current_char + ii))
        z_ascii=$(echo -n "z" | od -An -tuC)
        Z_ascii=$(echo -n "Z" | od -An -tuC)
        if [[ $current_ascii -eq $((z_ascii + 1)) ]] || [[ $current_ascii -eq $((Z_ascii + 1)) ]]; then
            if [[ $current_ascii -eq $((z_ascii + 1)) ]]; then
                plus="${plus}z"
            else
                plus="${plus}Z"
            fi
            ii=$((ii - 26))
            current_ascii=$((current_ascii - 26))
        fi
        hex_code=$(printf "%x" $((current_ascii)))
        char=$(printf "\\x$hex_code")
        new_filename="${source_filename%.*}${plus}${char}.${source_filename##*.}"
        cp "$source_filename" "$new_filename"
        ii=$((ii + 1))
    done
fi
```

Листинг окончательного варианта программы (Python):

```
import argparse


parser = argparse.ArgumentParser(usage='lab21 [OPTIONS] filename count start_char',
                                 description='''Generation of a set number of copies of the specified file.
                                 Copy names are generated by adding the next letter or number to the name of
                                 the source file, starting with the specified letter or number.
                                 ''')
parser.add_argument('source_filename', help='The source file to copy.')
parser.add_argument('count', help='Number of files to create.')
parser.add_argument('start_char', help='Starting letter or digit for file names.')
args = parser.parse_args()
source_filename = str(args.source_filename)
count = int(args.count)
start_char = args.start_char
# проверяю корректность введённых данных
if len(start_char) != 1 and not start_char.isdigit():
    print("Error: Starting letter or digit for file names must be a single character or a digit.")
    exit(1)
# проверяю существование файла для копирования и в случае успеха копирую его содержимое
try:
    with open(source_filename, 'r', encoding='utf-8') as file:
```

```
        file_text = file.read()
except FileNotFoundError:
    print(f"Error: file '{source_filename}' not found.")
    exit(1)
# определяю, по какому правилу должно задаваться имя, и создаю файл
plus = ''
ii = 0
for i in range(count):
    if start_char.isalpha():
        if ord(start_char) + ii == ord('z') + 1 or ord(start_char) + ii == ord('Z') + 1:
            if ord(start_char) + ii == ord('z') + 1:
                plus += 'z'
            else:
                plus += 'Z'
            ii -= 26
        new_filename = f"{source_filename[:source_filename.index('.')]}{plus}{chr(ord(start_char) +
ii)}{source_filename[source_filename.index('.'):]}"
        ii += 1
    else:
        new_filename = f"{source_filename[:source_filename.index('.')]}{int(start_char) +
i}{source_filename[source_filename.index('.'):]}"
    with open(new_filename, 'w', encoding='utf-8') as new_file:  # копирую содержимое файла
        new_file.write(file_text)
```

Распечатка протокола:

```
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh test1.txt 3 a
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh test1.txt 3 1
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh test1.txt 30 a
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh test1.txt 3 ab
Error: Starting letter or digit for file names must be a single character or a digit.
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh несущ.txt 3 a
Error: file 'несущ.txt' does not exist.
tan@DESKTOP-M1FO1D6:~$ bash lab21.sh -h
lab21 [OPTIONS] source_filename count start_char
Generation of a set number of copies of the specified file.
Copy names are generated by adding the next letter or number to the name of
the source file, starting with the specified letter or number.
Options:
  -h, --help    show this help message and exit
```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе лабораторной работы разработаны два скрипта (Bash и Python) для создания копий файла с автоматической генерацией имён путём добавления последовательных букв или цифр. Реализована корректная обработка переноса разрядов в буквенном режиме (после 'z' идёт 'za'). Скрипты включают проверку входных данных и обработку ошибок. Работа позволила закрепить навыки файлового ввода-вывода, обработки аргументов командной строки и строковых операций. Оба скрипта успешно протестированы в Ubuntu.

Подпись студента _____